Inside the Machine

Understanding Large Language Models

April 30, 2026

Contents

1 The Problem **1**
- 1.1 What does "predict" mean, precisely? 1
  - 1.1.1 Why condition on all previous words? 2
- 1.2 The chain rule: turning a hard problem into many small ones 2
- 1.3 What the model learns 3
- 1.4 The loss function: measuring how wrong the model is 4
  - 1.4.1 Surprise 4
  - 1.4.2 Entropy 5
  - 1.4.3 Cross-entropy: the training loss 5
  - 1.4.4 Perplexity 6
- 1.5 Python: seeing the loss in action 6

2 Tokens: The Atoms of Language **12**
- 2.1 Why not just use words? 12
- 2.2 The vocabulary 13
- 2.3 Byte-Pair Encoding (BPE): building a vocabulary from data 14
  - 2.3.1 The algorithm, step by step 14
  - 2.3.2 A worked example 15
  - 2.3.3 Applying the tokenizer to new text 16
- 2.4 What a token ID is 16
- 2.5 Python: building BPE from scratch 16
- 2.6 Why tokenization matters for the model 22

3 Embeddings: Giving Numbers to Meaning **23**
- 3.1 The impossibility of one-hot vectors 23

3.2 Vectors: a short, direct introduction 24
3.2.1 Length of a vector 24
3.2.2 The dot product 24
3.2.3 Cosine similarity: angle as similarity 25
3.3 The embedding: a point in space for every token . . . 26
3.3.1 How are the embeddings learned? 26
3.4 The geometry of meaning 27
3.4.1 Nearest neighbours 27
3.4.2 Analogy arithmetic 27
3.4.3 What does the embedding matrix look like? . 28
3.5 The softmax function 28
3.6 Python: exploring a real embedding space 29

4 Positional Encoding: Teaching the Model Where It Is 37
4.1 Why attention is order-blind 37
4.2 What we want from a positional encoding 38
4.3 Sine and cosine: the natural solution 38
4.3.1 A quick review of sine and cosine 38
4.3.2 Sinusoidal positional encoding 39
4.3.3 Why the dot product property matters 40
4.4 Learned positional embeddings 41
4.5 Rotary Positional Embedding (RoPE) 41
4.5.1 The 2D rotation idea 41
4.5.2 Extending to d dimensions 42
4.5.3 Why RoPE is the current standard 42
4.6 Python: seeing positional encodings 43

1 The Problem

After this chapter you will know exactly what a language model is trying to compute, what the training goal is, and why it is surprisingly simple to state, even if hard to solve.

Before we write a single equation, let's agree on what we are actually building.

A language model does one thing: it looks at a sequence of words (or pieces of words) and tries to predict what comes next. That's it. No deep magic. The entire 100-billion-parameter machine you hear about is, at its core, a very sophisticated next-word guesser.

But "predict what comes next" is a deceptively subtle idea. To make it precise—and to make it something we can actually calculate and improve—we need to give it a mathematical form. That is what this chapter does.

1.1 What does "predict" mean, precisely?

Suppose you are reading this sentence and you reach the blank: "The cat sat on the _____." You might guess *mat*, *floor*, *chair*, or *keyboard*. You would not guess *democracy* or *seventeen*. Some words are far more likely than others.

That is **probability**. A language model assigns a probability to every possible next word. The word with the highest probability is the model's best guess. But we keep all the probabilities, not just the

winner, because sometimes we want the model to be creative and pick a less-likely word on purpose (more on that in later chapters).

So if we call the words $w_1, w_2, w_3, \ldots$ (word one, word two, word three, ...), then the model's job is to compute:

$$P(\text{next word} \mid \text{all previous words})$$

The vertical bar "|" is read *given*. So this says: the probability of the next word, *given* all the words that came before it.

1.1.1 Why condition on all previous words?

Could we just look at the last word? Or the last two? Let's see why that would fail.

Context	What comes next?
...the bank	*robber? teller? account?*
...the river bank	*eroded? flooded?*
...the bank of the river	*was calm...*

The word "bank" can mean many things. To resolve the ambiguity, you need context from several words back—sometimes from the start of the paragraph. A model that only looks at the last word or two would constantly be confused. A model that looks at the entire history can disambiguate. This is one of the central reasons the Transformer architecture (which we build up over the next chapters) was such a big step forward: it can attend to the full history.

1.2 The chain rule: turning a hard problem into many small ones

There is one more thing the model needs to do. It doesn't just predict the next word—it assigns a probability to an *entire sequence* of words. Why? Because during training we need to measure how good the model is on complete sentences. And during generation, we are building a sequence word by word.

How do you assign a probability to a whole sequence? We use a fact from basic probability called the **chain rule**. We derive it from scratch here, no prior knowledge required.

Start with two events A and B. What is the probability that *both* happen? By the definition of conditional probability:

$$P(A \text{ and } B) \;=\; P(A) \cdot P(B \mid A)$$

This says: the probability that both A and B occur equals the probability of A alone, times the probability of B *assuming A already happened*.

Now extend to three events A, B, C:

$$P(A, B, C) \;=\; P(A) \cdot P(B \mid A) \cdot P(C \mid A, B)$$

And in general, for a sequence of words $w_1, w_2, \ldots, w_n$:

$$P(w_1, w_2, \ldots, w_n) \;=\; P(w_1) \cdot P(w_2 \mid w_1) \cdot P(w_3 \mid w_1, w_2) \;\cdots P(w_n \mid w_1, \ldots, w_{n-1})$$

This is *exact*—no approximation was made. We have taken the probability of a whole sentence and rewritten it as a product of next-word probabilities. Each factor asks: given everything before, what is the probability of this specific next word?

What this really means

The model reads left to right, one word at a time. At each step it asks: "given what I've seen so far, how likely is this next word?" Multiply all those likelihoods together and you get the probability of the whole sentence.

A grammatical, coherent sentence will have high probability (each word was a likely continuation). A random string of words will have very low probability (many of the steps were unlikely).

1.3 What the model learns

A language model is a machine that, when given $w_1, \ldots, w_{t-1}$, outputs a number between 0 and 1 for every possible word in the vocabulary. Those numbers must add up to 1 (one of them will be the next word). The model has millions of internal numbers called **parameters** that it adjusts during training to make its predictions better.

Training works like this:

1. Show the model a real sentence from the training data, say "The cat sat on the mat."
2. At each position, ask the model to predict the next word.
3. Measure how wrong the model was.
4. Adjust the parameters slightly to reduce that wrongness.
5. Repeat billions of times on billions of sentences.

Step 3—measuring how wrong the model was—requires a number. That number is the **loss function**. We will derive it carefully now.

1.4 The loss function: measuring how wrong the model is

1.4.1 Surprise

Imagine you are predicting coin flips. A fair coin gives you 50% heads. If it lands heads, you are mildly surprised—50% chance, so not shocking. But if someone hands you a coin you believe is always heads (100% probability), and it lands tails, you are *completely* shocked. Surprise scales with improbability.

Mathematically, the surprise at seeing event x with probability $p(x)$ is defined as:

$$\text{surprise}(x) = -\log_2 p(x)$$

Why the minus sign? Because $p(x)$ is between 0 and 1, and $\log_2$ of numbers between 0 and 1 is negative. The minus makes surprise a positive number. Why $\log_2$? Historically it was convenient (it gives surprise in *bits*), but any logarithm base works; the math is identical. We will use the natural logarithm ln from here on, which is standard in machine learning.

- If $p(x) = 1$ (you were certain): surprise $= -\ln 1 = 0$. No surprise at all.
- If $p(x) = 0.5$: surprise $= -\ln 0.5 \approx 0.69$.
- If $p(x) = 0.01$ (very unlikely): surprise $= -\ln 0.01 \approx 4.6$. Very surprising.

1.4.2 Entropy

Now suppose you have many possible outcomes, each with a probability. The average surprise is called **entropy**:

$$H = -\sum_{x} p(x) \ln p(x)$$

The $\sum$ symbol means "add up over all possible outcomes x." High entropy means the outcomes are roughly equally likely and the process is hard to predict. Low entropy means one outcome is very probable and the process is easy to predict.

1.4.3 Cross-entropy: the training loss

During training we have the *true* next word (from the text), and the model's predicted probability for that word. We want to penalise the model more when it assigns a low probability to the correct word.

If the correct next word is w and the model assigns it probability $p_\theta(w)$ (the subscript θ just means "the model with its current parameters θ"), then the loss on that one prediction is:

$$\ell = -\ln p_\theta(w)$$

This is the *cross-entropy loss* for a single prediction. If the model was very confident and correct ($p_\theta(w)$ close to 1), the loss is close to 0—good. If the model was very wrong ($p_\theta(w)$ close to 0), the loss is large—bad.

Over a whole sequence of n tokens, we average:

$$L(\theta) = -\frac{1}{n} \sum_{t=1}^{n} \ln p_\theta(w_t \mid w_1, \ldots, w_{t-1})$$

This is the number we minimise during training. Lower is better.

What this really means

The loss is just the average surprise the model feels when it sees the actual next word. Training makes the model less surprised by real text. A model that has trained on billions of sentences has learned to expect what real language looks like.

1.4.4 Perplexity

You will often see the word **perplexity** used to evaluate language models. It is just the exponent of the loss:

$$\text{Perplexity} = e^{L(\theta)}$$

If the loss is 0 (perfect prediction every time), perplexity is $e^0 = 1$. A perplexity of k roughly means the model is as confused as if it had to pick uniformly among k equally-likely options at each step. GPT-2 small achieves a perplexity around 35 on English text. State-of-the-art models today are below 10.

1.5 Python: seeing the loss in action

Here we use a small pretrained model to see these ideas concretely. You do not need to understand every line of code yet. Focus on the *output*: per-token surprise values that map directly to the formula above.

All code lives in the `code/` folder. Run with `python filename.py`

ch0_perplexity.py

```python
# One-time install (run in your terminal first):
#   pip install transformers torch numpy matplotlib

import torch
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from transformers import GPT2LMHeadModel, GPT2Tokenizer

# ── Load a small pretrained model ──────────────────────────
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model     = GPT2LMHeadModel.from_pretrained("gpt2")
model.eval()   # inference mode — no gradient tracking

def token_surprises(sentence):
    """
    Returns a list of (token_string, surprise) pairs.
    Surprise at position t = -ln P(w_t | w_1..w_{t-1}).
    """
    ids = tokenizer.encode(sentence, return_tensors="pt")
    with torch.no_grad():
```

```python
        # model outputs one logit vector per position
        logits = model(ids).logits   # shape: [1, seq_len, vocab_size]

    surprises = []
    for t in range(1, ids.shape[1]):
        # logits at position t-1 predict what comes at position t
        log_probs = torch.log_softmax(logits[0, t-1, :], dim=-1)
        prob_of_actual = log_probs[ids[0, t]].item()
        surprise = -prob_of_actual          # negate → positive number
        token_str = tokenizer.decode([ids[0, t]])
        surprises.append((token_str, surprise))

    return surprises

# ── Compare two sentences ──────────────────────────────
good = "The cat sat on the mat."
odd  = "The cat sat on the democracy."

surp_good = token_surprises(good)
surp_odd  = token_surprises(odd)

# Print the results
print("Grammatical sentence:")
for tok, s in surp_good:
    bar = "▇" * int(s)
    print(f"  {tok!r:20s}  surprise = {s:.2f}  {bar}")

print("\nSentence with odd ending:")
for tok, s in surp_odd:
    bar = "▇" * int(s)
    print(f"  {tok!r:20s}  surprise = {s:.2f}  {bar}")

# ── Plot both side by side ──────────────────────────────
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4),
                               sharey=True)
fig.suptitle("Per-token surprise  (−ln P)", fontsize=13)

for ax, surps, title in [
```

```python
        (ax1, surp_good, f'"{good}"'),
        (ax2, surp_odd,  f'"{odd}"')]:
    tokens = [t for t, _ in surps]
    values = [s for _, s in surps]
    colors = ["#e05c5c" if v > 3.5 else "#5c8ae0" for v in values]
    ax.bar(range(len(tokens)), values, color=colors, edgecolor="none")
    ax.set_xticks(range(len(tokens)))
    ax.set_xticklabels(tokens, rotation=40, ha="right", fontsize=9)
    ax.set_title(title, fontsize=9, pad=6)
    ax.set_ylabel("Surprise  -ln P")
    ax.spines[["top","right"]].set_visible(False)

hi = mpatches.Patch(color="#e05c5c", label="High surprise (> 3.5)")
lo = mpatches.Patch(color="#5c8ae0", label="Low surprise")
ax2.legend(handles=[lo, hi], loc="upper left", fontsize=8)

plt.tight_layout()
plt.savefig("figures/ch0_perplexity.pdf", bbox_inches="tight")
plt.show()
print("\nFigure saved to figures/ch0_perplexity.pdf")
```

Run this and look at the bar chart. The word "democracy" will have a dramatically taller bar than "mat" in the same sentence position. That tall bar is the model's surprise—and it is exactly the number added to the loss during training.

1. Change `good` to a different grammatical sentence. Do function words (*the, on*) have low surprise? Why?
2. Replace the odd word with something common but in the wrong place, e.g. "The cat sat on the running." Compare the surprise with "democracy".
3. Add a third sentence that is complete nonsense (random words). What happens to the average loss?

ch0_entropy.py

```python
# Visualise how entropy changes with n-gram context size.
# Shows: more context → lower entropy → easier prediction.

import math
import collections
import matplotlib.pyplot as plt

# A small corpus — paste more text here for better estimates
corpus = """
The quick brown fox jumps over the lazy dog.
A language model predicts the next word given all previous words.
The cat sat on the mat and looked at the moon.
She opened the door and walked into the garden.
He put the book on the table next to the lamp.
The sun rose slowly over the hills in the morning.
""".lower().split()

def ngram_entropy(tokens, n):
    """
    Estimate entropy of the n-th word given the (n-1) previous words.
    Uses relative frequency as a probability estimate.
    """
    context_counts = collections.defaultdict(int)
    ngram_counts   = collections.defaultdict(int)

    for i in range(len(tokens) - n):
        ctx   = tuple(tokens[i : i+n-1])   # the (n-1) context words
        word  = tokens[i + n - 1]           # the n-th word
        context_counts[ctx] += 1
        ngram_counts[(ctx, word)] += 1

    # For each context, compute the conditional distribution,
    # then average its entropy
    total_entropy = 0.0
    total_weight  = 0

    for ctx, ctx_count in context_counts.items():
        # Find all words that follow this context
        followers = {w: ngram_counts[(ctx, w)]
```

```python
                    for w in set(v for (c,v) in ngram_counts if c == ctx)}
        # Conditional probabilities
        probs = [count / ctx_count for count in followers.values()]
        # Entropy of this conditional distribution
        h = -sum(p * math.log(p) for p in probs if p > 0)
        total_entropy += h * ctx_count
        total_weight  += ctx_count

    return total_entropy / total_weight if total_weight > 0 else 0

ns = [1, 2, 3, 4]
entropies = [ngram_entropy(corpus, n) for n in ns]
labels = ["unigram\n(no context)",
          "bigram\n(1 word context)",
          "trigram\n(2 word context)",
          "4-gram\n(3 word context)"]

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(ns, entropies, "o-", color="#2C3E6B", linewidth=2,
        markersize=8, markerfacecolor="white", markeredgewidth=2)
for n, h, lbl in zip(ns, entropies, labels):
    ax.annotate(f"{h:.2f}", xy=(n, h), xytext=(0, 10),
                textcoords="offset points", ha="center", fontsize=9)
ax.set_xticks(ns)
ax.set_xticklabels(labels, fontsize=9)
ax.set_ylabel("Conditional entropy (nats)")
ax.set_title("More context → lower entropy → easier to predict",
             fontsize=11)
ax.spines[["top","right"]].set_visible(False)
ax.set_ylim(0, None)
plt.tight_layout()
plt.savefig("figures/ch0_entropy.pdf", bbox_inches="tight")
plt.show()
print("Figure saved to figures/ch0_entropy.pdf")
```

When you run this, you will see entropy drop as you give the model more context. This is the empirical justification for conditioning on

the full history.

Summary

- A language model computes $P(w_t \mid w_1, \dots, w_{t-1})$—the probability of the next word given all previous words.
- The chain rule lets us factor the probability of a whole sentence into a product of next-word probabilities.
- We train the model by minimising the cross-entropy loss: $L = -\frac{1}{n} \sum_t \ln p_\theta(w_t \mid \text{context})$.
- Perplexity is e^L—a human-readable score of how confused the model is.
- Everything else in this book is about *how* we build a model to compute this probability well.

2 Tokens: The Atoms of Language

After this chapter you will understand what a token is, why words are not the right unit, how the Byte-Pair Encoding algorithm builds a vocabulary from raw text, and why the choice of tokenizer quietly shapes everything the model can do.

In Chapter 0 we used the word "word" loosely. Now we have to be more careful. Language models do not actually work with words. They work with **tokens**—chunks of text that may be whole words, parts of words, punctuation marks, or even single letters. This chapter explains why, and how tokens are produced.

2.1 Why not just use words?

Your first instinct might be: split the text on spaces and call each piece a word. That seems natural. But it runs into several immediate problems.

Problem 1: vocabulary explosion. The English language has hundreds of thousands of distinct word forms. "run", "runs", "ran", "running", "runner" are all different strings. A model that treats each as a separate unrelated item must learn about each one from scratch. The vocabulary would need to hold every conjugation, declension, and compound form. For a single language this might be manageable; for a multilingual model it becomes enormous.

Problem 2: numbers and code. What about "`3.14159`"? Or "`def calculate_loss`"? These are not dictionary words. A word-level tokenizer would either refuse to handle them or create a vocabulary so large it is unusable.

Problem 3: novel words. New words appear constantly. A word-level model trained yesterday has never seen "ChatGPT" or the name of yesterday's newly discovered exoplanet. It has no representation for them at all.

The solution used by all modern language models is to work at an intermediate level between letters and words. Short, frequent pieces of text get their own token; rare words get split into meaningful sub-pieces that the model can still reason about.

2.2 The vocabulary

A **vocabulary** V is simply a finite set of strings. Each string in the vocabulary is called a token. Every piece of text the model will ever see must be representable as a sequence of tokens from V.

The vocabulary has a fixed size, written $|V|$ (read: "the size of V"). Common sizes:

- GPT-2: $|V| = 50{,}257$
- GPT-4 / ChatGPT: $|V| = 100{,}277$
- LLaMA 3: $|V| = 128{,}000$

Larger vocabularies mean each token covers more text (good for efficiency) but require more memory and make the model's final prediction step harder (it must assign a probability to more options).

The process of turning raw text into a sequence of tokens is called **tokenization**. The machine that does it is a **tokenizer**. A good tokenizer is:

1. **Invertible**: you can always recover the original text from the token sequence. No information is lost.

2. **Efficient**: common pieces of text get single tokens; nothing is needlessly split.
3. **General**: it can handle any string, not just dictionary words.

2.3 Byte-Pair Encoding (BPE): building a vocabulary from data

The most widely used tokenization method is **Byte-Pair Encoding**, or BPE. The key idea is beautiful: *start with the smallest possible units (individual bytes, or letters) and repeatedly merge the most common adjacent pair into a new token*. Run enough merges and you end up with a vocabulary full of common words and word-pieces.

BPE was originally a text compression algorithm (1994). It was adapted for NLP tokenization in 2016 and is now the dominant method.

2.3.1 The algorithm, step by step

Here is BPE laid out precisely.

Algorithm 1: Byte-Pair Encoding — vocabulary construction

Input: A text corpus C; target vocabulary size $|V_{\text{target}}|$
Output: A vocabulary V and a list of merge rules

```
# Step 1: Initialise with individual characters (or
  bytes)
```

$V \leftarrow$ {all unique characters in C} $\cup$ {`</w>`};
Represent every word in C as a sequence of its characters;

```
# Step 2: Repeatedly merge the most frequent pair
```

while $|V| < |V_{target}|$ **do**
  Count every adjacent pair of symbols in the current representation of C;
  Let (a, b) be the pair with the highest count;
  Create new symbol ab by concatenating a and b;
  Add ab to V;
  Record merge rule: $(a, b) \rightarrow ab$;
  Replace every occurrence of adjacent (a, b) in C with ab;
end
return V, *list of merge rules*

2.3.2 A worked example

Let's trace through BPE on a tiny corpus. Our corpus (after counting word frequencies) is:

```
low: 5,  lower: 2,  newest: 6,  widest: 3
```

We represent each word as a space-separated character sequence, with a special end-of-word marker `</w>`:

Word	Initial representation
low (×5)	`l o w </w>`
lower (×2)	`l o w e r </w>`
newest (×6)	`n e w e s t </w>`
widest (×3)	`w i d e s t </w>`

Iteration 1. Count all adjacent pairs:

- `(e, s)`: appears in "newest" (×6) and "widest" (×3) = 9 times
- `(e, w)`: "newest" (×6) = 6 times
- `(l, o)`: "low" (×5) + "lower" (×2) = 7 times
- ...

The winner is `(e, s)` with count 9. Merge: `e` + `s` → `es`. Now "newest" becomes `n e w es t </w>` and "widest" becomes `w i d es t </w>`.

Iteration 2. Count again. Now `(es, t)` appears 9 times. Merge: `es` + `t` → `est`.

This continues. After enough merges, common words like "low" and "new" will become single tokens, while rare words will be split into meaningful pieces like "new" + "est" = "newest".

What this really means

BPE is compression by committee. Pieces of text that appear together often get promoted to a single token. By the time you are done, the vocabulary has naturally captured words, common word endings (*-ing*, *-est*, *-ed*), punctuation patterns, and even common multi-word chunks like "New York" in some models.

2.3.3 Applying the tokenizer to new text

Once training is done, we have a fixed list of merge rules. To tokenize new text, we apply those rules in order (the same order they were learned), left to right. The most frequent merges happen first, so common words collapse to a single token quickly.

2.4 What a token ID is

After tokenization, each token is replaced by an integer—its **token ID**. These are just indices into the vocabulary list. The vocabulary is stored as a dictionary: token string → integer.

So the sentence "Hello world" might become `[15496, 995]`, and the integer 15496 is what gets fed into the model. The model never sees the letters. It sees integers.

In the next chapter we will see how those integers get turned into meaningful numbers that the model can do math with. For now just note: *a token is a string; a token ID is an integer; the model works with IDs*.

2.5 Python: building BPE from scratch

ch1_bpe_scratch.py

```python
# Build a BPE tokenizer from scratch.
# No external libraries needed — only Python's standard library.

import re
import collections

# ── Corpus and helper ──────────────────────────────────────
corpus = [
    "low low low low low",
    "lower lower",
    "newest newest newest newest newest newest",
    "widest widest widest",
]

def get_vocab(corpus):
```

```python
    """
    Convert corpus into a dict mapping
    'space-separated character sequence' -> count.
    Each word gets a </w> marker at the end.
    """
    vocab = collections.defaultdict(int)
    for line in corpus:
        for word in line.strip().split():
            # e.g. 'low' -> 'l o w </w>'
            vocab[" ".join(list(word)) + " </w>"] += 1
    return dict(vocab)

def get_pair_counts(vocab):
    """Count every adjacent pair of symbols across all words."""
    pairs = collections.defaultdict(int)
    for word, count in vocab.items():
        symbols = word.split()
        for i in range(len(symbols) - 1):
            pairs[(symbols[i], symbols[i+1])] += count
    return pairs

def merge_vocab(pair, vocab):
    """Replace all occurrences of 'a b' with 'ab' in the vocab."""
    out = {}
    bigram = " ".join(pair)           # e.g. 'e s'
    replacement = "".join(pair)       # e.g. 'es'
    for word, count in vocab.items():
        new_word = word.replace(bigram, replacement)
        out[new_word] = count
    return out

# ── Run BPE ──────────────────────────────────────────────
vocab = get_vocab(corpus)
num_merges = 10

print("Starting vocabulary (each word as characters):")
for word, count in vocab.items():
    print(f"  {count}×  {word}")
print()

```

```python
merge_rules = []

for i in range(num_merges):
    pair_counts = get_pair_counts(vocab)
    if not pair_counts:
        break
    best_pair = max(pair_counts, key=pair_counts.get)
    count     = pair_counts[best_pair]

    vocab = merge_vocab(best_pair, vocab)
    merge_rules.append(best_pair)

    new_token = "".join(best_pair)
    print(f"Merge {i+1:2d}: {best_pair[0]!r} + {best_pair[1]!r} "
          f"-> {new_token!r}  (seen {count} times)")

print("\nFinal vocabulary state:")
for word, count in sorted(vocab.items(), key=lambda x: -x[1]):
    print(f"  {count}×  {word}")

print("\nMerge rules (applied in order to tokenize new text):")
for i, rule in enumerate(merge_rules, 1):
    print(f"  {i:2d}.  {rule[0]} + {rule[1]}  -> {''.join(rule)}")
```

Run this and watch the merge steps. You will see "`e s`" merge into "`es`" first, then "`es t`" into "`est`", and so on. This is the algorithm that GPT-4 uses—just run on a trillion characters instead of a few toy words.

ch1_tiktoken.py

```python
# Use the real GPT-4 tokenizer to explore tokenization.
# Install first:  pip install tiktoken

import tiktoken

# Load the GPT-4 tokenizer
enc = tiktoken.get_encoding("cl100k_base")

def show_tokens(text, enc):
```

```python
    ids    = enc.encode(text)
    pieces = [enc.decode([i]) for i in ids]
    print(f"\nText: {text!r}")
    print(f"Token count: {len(ids)}")
    print("Tokens:")
    for piece, tid in zip(pieces, ids):
        print(f"  {tid:6d}  {piece!r}")
    return ids

# ── English prose ──────────────────────────────────────────
show_tokens("The transformer model learns to predict.", enc)

# ── Python code ────────────────────────────────────────────
show_tokens("def cross_entropy(p, q):\n    return -sum(p*log(q))", enc)

# ── A word with many suffixes ──────────────────────────────
show_tokens("tokenization tokenizing tokenizer tokenize", enc)

# ── Numbers ────────────────────────────────────────────────
show_tokens("The year 1994. Pi = 3.14159265.", enc)

# ── Efficiency: compare English vs non-English text ────────
texts = {
    "English":  "The cat sat on the mat in the garden.",
    "Japanese": "□□□□□□□□□□□□□□",
    "Python":   "for i in range(len(tokens)):\n    loss += -log(p[i])",
}

print("\n── Compression comparison ──────────────────────────")
print(f"{'Language/type':<14} {'chars':>6} {'tokens':>7} "
      f"{'chars/token':>12}")
print("-" * 46)
for label, text in texts.items():
    ids = enc.encode(text)
    ratio = len(text) / len(ids)
    print(f"{label:<14} {len(text):>6} {len(ids):>7} {ratio:>12.2f}")

print("\nHigher chars/token = more efficient tokenization.")
```

```python
print("Notice how Japanese uses far more tokens per character")
print("because the tokenizer was trained mostly on English.")
```

1. Run `ch1_tiktoken.py`. How many tokens does the word "tokenization" produce? What about "token"?
2. Try tokenizing a word that does not exist, like "flurbenschaft". What pieces does BPE split it into?
3. Switch the encoding from `"cl100k_base"` to `"r50k_base"` (GPT-2's tokenizer). Does the token count change? Which is more efficient?
4. Tokenize a 10-digit number like "3141592653". How many tokens? Why might this be a problem for arithmetic?

ch1_fertility.py

```python
# Visualise tokenizer "fertility" (tokens per word) across languages.
# High fertility = the tokenizer is inefficient for that language.
# Install first:  pip install tiktoken matplotlib

import tiktoken
import matplotlib.pyplot as plt

enc = tiktoken.get_encoding("cl100k_base")

# Sample sentences (roughly the same semantic content)
samples = {
    "English":    "The cat sat on the mat next to the window.",
    "Spanish":    "El gato se sentó en la alfombra junto a la ventana.",
    "French":     "Le chat s'est assis sur le tapis près de la fenêtre.",
    "German":     "Die Katze saß auf der Matte neben dem Fenster.",
    "Japanese":   "□□□
```

```python
    "Python code":"cat.sit(location=mat, near='window')",
}

results = {}
for lang, text in samples.items():
    words  = len(text.split())
    tokens = len(enc.encode(text))
    results[lang] = tokens / max(words, 1)

# Sort by fertility
results = dict(sorted(results.items(), key=lambda x: x[1]))

fig, ax = plt.subplots(figsize=(8, 4.5))
colors = ["#5c8ae0" if v < 2 else "#e0855c" for v in results.values()]
bars = ax.barh(list(results.keys()), list(results.values()),
               color=colors, edgecolor="none", height=0.55)

ax.axvline(1.0, color="#aaa", linestyle="--", linewidth=1)
for bar, val in zip(bars, results.values()):
    ax.text(val + 0.03, bar.get_y() + bar.get_height()/2,
            f"{val:.2f}", va="center", fontsize=9)

ax.set_xlabel("Tokens per word  (lower = more efficient)")
ax.set_title("GPT-4 tokenizer fertility by language\n"
             "(trained mostly on English → English is most efficient)",
             fontsize=10)
ax.spines[["top","right"]].set_visible(False)
plt.tight_layout()
plt.savefig("figures/ch1_fertility.pdf", bbox_inches="tight")
plt.show()
print("Figure saved to figures/ch1_fertility.pdf")
```

The bar chart will make a quiet but important point visible: the tokenizer is not neutral. It was trained on mostly English text, so English gets efficient single-token representations for common words. Japanese and Arabic need far more tokens for the same content. This means non-English text uses more of the model's limited context window and effectively costs more to process.

2.6 Why tokenization matters for the model

Once you understand tokens, a few things about language models that seemed mysterious become clear.

Context window. When people say a model has a "context window of 128,000 tokens", they mean it can look back at that many tokens when predicting the next one. In English, this is roughly 100,000 words. But in Japanese, the same number of concepts might use 200,000 tokens (because each word takes more tokens). So the effective context window is shorter for Japanese text.

Arithmetic. Numbers like 1,234,567 get split into several tokens: "1", "," "234", ","“567" or similar, depending on the tokenizer. The model must learn to reason about numbers from fragments—which is genuinely hard. This is one reason language models sometimes struggle with mental arithmetic.

Rare words and names. A name like "Schwarzenegger" gets split into several tokens: "Schw", "ar", "zen", "egg", "er" (or similar). The model can still use context to understand who is being discussed, but it has to piece the name together from parts.

Summary

- A **token** is a piece of text—part of a word, a whole word, or punctuation—that the model treats as a single unit.
- The **vocabulary** V is the fixed set of all tokens the model knows. Size: typically 32,000–128,000.
- **BPE** builds the vocabulary from data by repeatedly merging the most frequent adjacent pair of symbols.
- Each token is stored as an integer **token ID**.
- The tokenizer is not language-neutral: languages with less training data get split more finely and use more tokens for the same content.
- In the next chapter we will see how token IDs—plain integers—get turned into rich geometric objects the model can reason about.

3 Embeddings: Giving Numbers to Meaning

After this chapter you will understand why we represent tokens as points in space, what an inner product is and why it measures similarity, how training shapes the geometry of those points, and how to explore a real embedding space in Python.

At the end of Chapter 1 we had token IDs: plain integers like `15496` or `995`. The model cannot do anything useful with bare integers. You cannot meaningfully add, subtract, or multiply them. The fact that "cat" has ID 3797 and "dog" has ID 3290 tells you absolutely nothing about whether those words are related.

We need a richer representation—one where the mathematical structure mirrors the structure of language. This chapter builds that representation from the ground up.

3.1 The impossibility of one-hot vectors

The first idea you might have is this: create a list of length $|V|$ (say, 50,000), put a 1 in the slot for the current token, and zeros everywhere else. This is called a **one-hot vector**.

$$\text{“cat”} \;\mapsto\; \underbrace{(0, 0, \ldots, 0, 1, 0, \ldots, 0)}_{50{,}000 \text{ entries}}$$

The problem: every token is equidistant from every other. The distance from "cat" to "dog" is exactly the same as the distance from "cat" to "democracy". There is no notion of similarity.

More concretely: every one-hot vector has length 1, and the angle between any two different one-hot vectors is exactly 90 degrees. The space has no structure at all—it is just a vast empty room where each word stands in its own corner, as far from every other word as it possibly can be.

We need a different kind of representation.

3.2 Vectors: a short, direct introduction

If you are comfortable with vectors and dot products, skim this section and start at §2.3.

A **vector** is an ordered list of real numbers. We write it as:

$$\mathrm{v} = (v_1, v_2, \dots, v_d)$$

where d is the number of entries, called the **dimension**. Each entry v_i is just a number—positive, negative, or zero.

You can picture a 2-dimensional vector as an arrow from the origin to a point in a plane. A 3-dimensional vector is an arrow to a point in space. A 512-dimensional vector is the same idea—just in a space we cannot draw, but can still calculate in.

3.2.1 Length of a vector

The length (or **norm**) of a vector is what you would expect from the Pythagorean theorem extended to many dimensions:

$$\|\mathrm{v}\| \;=\; \sqrt{v_1^2 + v_2^2 + \cdots + v_d^2}$$

For example, $\mathrm{v} = (3, 4)$ has length $\sqrt{9 + 16} = \sqrt{25} = 5$.

3.2.2 The dot product

Given two vectors u and v of the same dimension, their **dot product** is:

$$\mathrm{u} \cdot \mathrm{v} \;=\; u_1 v_1 + u_2 v_2 + \cdots + u_d v_d \;=\; \sum_{i=1}^{d} u_i v_i$$

You multiply corresponding entries and add them up. The result is a single number, not a vector.

Example. $u = (1, 2, 3)$, $v = (4, -1, 2)$:

$$u \cdot v = (1)(4) + (2)(-1) + (3)(2) = 4 - 2 + 6 = 8$$

Why does the dot product matter? There is a geometric fact connecting it to angle. If θ is the angle between u and v, then:

$$u \cdot v \;=\; \|u\| \cdot \|v\| \cdot \cos\theta$$

We will not prove this from scratch here (it follows from the law of cosines), but let's understand what it implies. If two vectors point in the same direction, $\theta = 0$ and $\cos\theta = 1$, so the dot product is as large as possible. If they are perpendicular, $\theta = 90°$ and $\cos\theta = 0$, so the dot product is zero. If they point in opposite directions, $\theta = 180°$ and $\cos\theta = -1$, so the dot product is negative.

3.2.3 Cosine similarity: angle as similarity

We want a measure of similarity between two vectors that is between -1 and 1 and does not depend on the vectors' lengths. Rearranging the equation above:

$$\cos\theta \;=\; \frac{u \cdot v}{\|u\| \cdot \|v\|}$$

This is called **cosine similarity**. It measures the angle between two vectors, ignoring how long they are. Values:

- $+1$: same direction (maximally similar)
- 0: perpendicular (unrelated)
- -1: opposite directions (maximally dissimilar)

What this really means

Two vectors are similar if they *point in the same direction*. Length does not matter—only angle. Think of two people each pointing at the same mountain from different distances. They are both pointing in the same direction, even though one is close and one is far.

3.3 The embedding: a point in space for every token

An **embedding** assigns a d-dimensional vector to each token. We store all these vectors in a matrix W_E with dimensions $|V| \times d$. Row i of this matrix is the embedding vector for token i.

$$W_E \in \mathbb{R}^{|V| \times d}$$

$|V|$: vocabulary size (e.g. 50,000). d: embedding dimension (e.g. 512, 768, 4096).
The embedding of token i is the i-th row of W_E.

d is always *much* smaller than $|V|$. Instead of 50,000 dimensions (one per possible token), we use maybe 768. This forces the model to represent tokens in a compressed, shared space where structure must emerge. Tokens that appear in similar contexts end up close together; tokens that never appear in similar contexts end up far apart.

3.3.1 How are the embeddings learned?

The embedding matrix W_E starts as random numbers. During training, every time the model makes a wrong prediction, the loss is calculated and gradients flow back through the entire model—including back to W_E. The entries of the matrix get updated slightly to reduce the loss.

Over billions of training steps, on billions of sentences, the updates accumulate. The gradient for token "cat" is influenced by all the sentences in which "cat" appeared. If "cat" and "dog" appear in similar grammatical positions and with similar surrounding words, their gradients will push their vectors in similar directions. Over time their vectors converge to similar locations in space.

This is the **distributional hypothesis**: words that appear in similar contexts tend to have similar meanings. The training process operationalises this ancient linguistic observation into geometry.

3.4 The geometry of meaning

Once embeddings are trained, the geometry becomes remarkably rich.

3.4.1 Nearest neighbours

The tokens with the highest cosine similarity to "cat" tend to be animal names: "dog", "kitten", "puppy", "rabbit". Not because we told the model what "cat" means, but because "cat" and "dog" appear in statistically similar positions in billions of sentences.

3.4.2 Analogy arithmetic

One of the striking early discoveries about word embeddings (from the Word2Vec paper, 2013) was that *arithmetic on vectors mirrors analogical reasoning*. The classic example:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

Why does this work? Imagine there is a direction in the embedding space that corresponds to the concept of "gender" — call it g. And a separate direction for "royalty". Then:

$$\begin{aligned}
\text{king} &= (\text{some royalty}) + (\text{male direction}) \\
\text{queen} &= (\text{some royalty}) + (\text{female direction}) \\
\text{man} &= (\text{some baseline}) + (\text{male direction}) \\
\text{woman} &= (\text{some baseline}) + (\text{female direction})
\end{aligned}$$

So `king` − `man` cancels the male direction, leaving just the royalty. Adding `woman` reinstates the female direction in its place, arriving at `queen`.

What this really means

The embedding space is not a random fog of points. It has *structure*: certain directions in the space correspond to human-interpretable concepts like gender, tense, singular/plural, country/capital. The model learns this structure purely from patterns in text—no human ever labeled a "gender direction". It emerges on its own from training.

3.4.3 What does the embedding matrix look like?

The matrix W_E is large. For GPT-2 small: $50{,}257 \times 768$. That is about 38 million numbers. Despite its size, most of the *useful* structure lives in a much lower-dimensional subspace. If you compute the singular values of W_E (a measure of how much variance each dimension carries), you find that the first 100–200 singular values dominate, and the rest are much smaller. The effective dimensionality of the learned embeddings is far lower than 768.

3.5 The softmax function

Softmax appears here in the context of embeddings; it will appear again in attention (Chapter 4) and in the final output layer.

There is one more mathematical object we need before the Python section. At the end of the model, we need to turn the model's internal numbers into probabilities over the vocabulary. Probabilities must be:

- All non-negative (no negative probabilities).
- Sum to exactly 1.

The **softmax function** does exactly this. Given a vector of n arbitrary real numbers $\mathrm{z} = (z_1, z_2, \ldots, z_n)$, softmax outputs:

$$\mathrm{softmax}(\mathrm{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

The denominator adds up all the exponentials, so that each entry of the output is a fraction between 0 and 1, and all entries sum to exactly 1.

Why exponentials? The exponential function e^x has two useful properties: it is always positive (so negative inputs become small positive outputs, not negative ones), and it grows very fast for large inputs. This means the entry with the largest z_i gets a disproportionately large probability—the softmax tends to concentrate probability on the maximum, while still being a smooth, differentiable function.

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}} \qquad \text{outputs sum to 1, all entries in } (0,1)$$

What this really means

Softmax turns a list of raw scores (called *logits*) into a probability distribution. The largest score gets most of the probability. If you multiply all inputs by a large number (a high *temperature*), the distribution becomes more peaked—one winner takes almost everything. If you multiply by a small number (low temperature), the distribution flattens and the model becomes more creative.

3.6 Python: exploring a real embedding space

ch2_cosine.py

```python
# Load real GloVe word embeddings and explore cosine similarity.
# Install: pip install numpy matplotlib requests

import numpy as np
import os, urllib.request, zipfile

# ── Download GloVe 50d (once) ──────────────────────────────
URL = "https://nlp.stanford.edu/data/glove.6B.zip"
ZIP = "glove.6B.zip"
TXT = "glove.6B.50d.txt"

if not os.path.exists(TXT):
    if not os.path.exists(ZIP):
        print("Downloading GloVe 50d (~170 MB)... please wait")
        urllib.request.urlretrieve(URL, ZIP)
    print("Extracting...")
    with zipfile.ZipFile(ZIP) as z:
        z.extract(TXT)
    print("Done.")

```

```python
# ── Load into a dict: word -> numpy array ──────────────────
print("Loading vectors...")
glove = {}
with open(TXT, encoding="utf-8") as f:
    for line in f:
        parts = line.split()
        word  = parts[0]
        vec   = np.array([float(x) for x in parts[1:]])
        glove[word] = vec
print(f"Loaded {len(glove):,} word vectors of dimension {len(next(iter(glove.values())))}.")

# ── Cosine similarity ──────────────────────────────────────
def cosine(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def nearest(word, n=8):
    if word not in glove:
        print(f"'{word}' not in vocabulary")
        return []
    target = glove[word]
    sims = [(w, cosine(target, v)) for w, v in glove.items() if w != word]
    sims.sort(key=lambda x: -x[1])
    return sims[:n]

# ── Find nearest neighbours ────────────────────────────────
for query in ["cat", "king", "python", "terrible"]:
    print(f"\nNearest to '{query}':")
    for w, s in nearest(query):
        bar = "▇" * int(s * 20)
        print(f"  {w:<15}  cos = {s:.3f}  {bar}")
```

ch2_analogy.py

```python
# Vector arithmetic for analogical reasoning.
# Requires glove loaded as in ch2_cosine.py — paste that loading
# code above this, or import it.

# Analogy: a is to b as c is to ?
# Answer: find the word closest to (b - a + c)
```

```python

def analogy(a, b, c, n=5, exclude=None):
    """
    a : b :: c : ?
    Returns the n nearest words to vec(b) - vec(a) + vec(c).
    """
    if any(w not in glove for w in [a, b, c]):
        missing = [w for w in [a,b,c] if w not in glove]
        print(f"Words not in vocab: {missing}")
        return []
    target = glove[b] - glove[a] + glove[c]
    skip   = {a, b, c} | (set(exclude) if exclude else set())
    sims   = [(w, cosine(target, v))
              for w, v in glove.items() if w not in skip]
    sims.sort(key=lambda x: -x[1])
    return sims[:n]

# Classic analogies
tests = [
    ("man",    "king",   "woman"),    # -> queen
    ("paris",  "france", "berlin"),   # -> germany
    ("slow",   "slower", "fast"),     # -> faster
    ("good",   "better", "bad"),      # -> worse
    ("cat",    "cats",   "dog"),      # -> dogs
]

for a, b, c in tests:
    print(f"\n{a} : {b}  ::  {c} : ?")
    for w, s in analogy(a, b, c):
        print(f"  {w:<15}  {s:.3f}")
```

ch2_pca_vis.py

```python
# Project 200 words into 2D using PCA and plot them.
# Shows that semantically related words cluster together.
# Install: pip install numpy matplotlib scikit-learn

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from sklearn.decomposition import PCA

```

```python
# (Assumes glove dict already loaded — paste loading block from
#  ch2_cosine.py above this section)

# ── Words to visualise, grouped by category ──────────────
groups = {
    "animals":   ["cat","dog","horse","rabbit","fish","bird","wolf","bear"],
    "countries": ["france","germany","japan","china","brazil","egypt","india"],
    "royalty":   ["king","queen","prince","princess","throne","crown","royal"],
    "verbs":     ["run","walk","sleep","eat","think","write","speak","jump"],
    "numbers":   ["one","two","three","four","five","six","seven","eight"],
}
colors = {
    "animals":   "#e05c5c",
    "countries": "#5c8ae0",
    "royalty":   "#9c5ce0",
    "verbs":     "#5ce0a0",
    "numbers":   "#e0b05c",
}

# Collect words and their embeddings
all_words = []
all_vecs  = []
all_cols  = []

for group, words in groups.items():
    for w in words:
        if w in glove:
            all_words.append(w)
            all_vecs.append(glove[w])
            all_cols.append(colors[group])

X = np.array(all_vecs)

# ── PCA: reduce from 50d to 2d ────────────────────────────
pca   = PCA(n_components=2)
X_2d  = pca.fit_transform(X)
```

```python
var_explained = pca.explained_variance_ratio_

# ── Plot ──────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(9, 7))
ax.scatter(X_2d[:, 0], X_2d[:, 1],
           c=all_cols, s=40, alpha=0.85, linewidths=0)

for i, word in enumerate(all_words):
    ax.annotate(word, (X_2d[i, 0], X_2d[i, 1]),
                fontsize=7.5, alpha=0.8,
                xytext=(3, 3), textcoords="offset points")

# Legend
handles = [mpatches.Patch(color=c, label=g)
           for g, c in colors.items()]
ax.legend(handles=handles, loc="upper right", fontsize=8,
          framealpha=0.8)

ax.set_xlabel(f"PC1  ({var_explained[0]*100:.1f}% variance)", fontsize=9)
ax.set_ylabel(f"PC2  ({var_explained[1]*100:.1f}% variance)", fontsize=9)
ax.set_title("GloVe 50d embeddings projected to 2D via PCA\n"
             "Semantically related words cluster together",
             fontsize=10)
ax.spines[["top","right"]].set_visible(False)
plt.tight_layout()
plt.savefig("figures/ch2_pca.pdf", bbox_inches="tight")
plt.show()
print("Figure saved to figures/ch2_pca.pdf")
```

When you run this, look for three things in the scatter plot:

- The country names should cluster together, separate from the animals.
- The royalty words should cluster near each other.
- "king" and "queen" should be close, and both should be near the royalty cluster and slightly separated from the plain noun cluster.

All of this structure emerged from training on raw text—no human drew the map.

ch2_embedding_matrix.py

```python
# Load GPT-2's embedding matrix and examine its structure.
# Shows: most variance lives in ~100 directions, not all 768.
# Install: pip install transformers torch matplotlib numpy

import torch
import numpy as np
import matplotlib.pyplot as plt
from transformers import GPT2Model

print("Loading GPT-2 small...")
model = GPT2Model.from_pretrained("gpt2")
model.eval()

# Extract the embedding matrix: shape [50257, 768]
W_E = model.wte.weight.detach().numpy()
print(f"Embedding matrix shape: {W_E.shape}")
print(f"  {W_E.shape[0]:,} tokens  ×  {W_E.shape[1]} dimensions")
print(f"  Total parameters: {W_E.size:,}")

# ── Compute singular values ──────────────────────────────
# SVD: W_E = U Σ Vᵀ
# The singular values Σ tell us how much variance each direction holds.
print("\nComputing singular value decomposition...")
_, singular_values, _ = np.linalg.svd(W_E, full_matrices=False)

# Cumulative variance explained
total_var = np.sum(singular_values ** 2)
cum_var   = np.cumsum(singular_values ** 2) / total_var

# Find how many dimensions to explain 80%, 90%, 95%
for target in [0.80, 0.90, 0.95]:
    k = np.searchsorted(cum_var, target) + 1
    print(f"  {target*100:.0f}% of variance explained by top {k} dimensions "
          f"(out of {W_E.shape[1]})")

# ── Plot singular values ──────────────────────────────
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

```

```python
ax1.plot(singular_values[:200], color="#2C3E6B", linewidth=1.5)
ax1.set_xlabel("Singular value rank")
ax1.set_ylabel("Singular value magnitude")
ax1.set_title("Singular values of GPT-2 embedding matrix\n"
              "(first 200 shown)")
ax1.fill_between(range(200), singular_values[:200],
                 alpha=0.15, color="#2C3E6B")
ax1.spines[["top","right"]].set_visible(False)

ax2.plot(cum_var[:300] * 100, color="#9A6B00", linewidth=1.5)
ax2.axhline(80, color="#aaa", linestyle="--", linewidth=0.8)
ax2.axhline(90, color="#aaa", linestyle="--", linewidth=0.8)
ax2.axhline(95, color="#aaa", linestyle="--", linewidth=0.8)
ax2.set_xlabel("Number of dimensions used")
ax2.set_ylabel("Cumulative variance explained (%)")
ax2.set_title("Cumulative variance explained\n"
              "(first 300 dimensions)")
ax2.spines[["top","right"]].set_visible(False)

plt.suptitle("GPT-2 embedding matrix: most structure is low-dimensional",
             fontsize=11, y=1.02)
plt.tight_layout()
plt.savefig("figures/ch2_svd.pdf", bbox_inches="tight")
plt.show()
print("\nFigure saved to figures/ch2_svd.pdf")
```

The left panel shows a classic "scree plot": the singular values drop steeply at first and then flatten. The steep part is where the useful structure is. The flat part is mostly noise. The right panel shows that you can explain 90% of the variance with fewer than 200 dimensions, even though the matrix officially has 768.

1. In `ch2_pca_vis.py`, change PCA to t-SNE by replacing the PCA lines with:
   ```
   from sklearn.manifold import TSNE
   X_2d = TSNE(n_components=2, perplexity=15).fit_transform(X)
   ```
   Does more structure become visible? t-SNE preserves local

structure (nearby points stay nearby) but distorts global distances.

2. Add a new group to `ch2_pca_vis.py` for professions: "doctor", "nurse", "engineer", "teacher", "lawyer". Where does this cluster land relative to the others?
3. In `ch2_analogy.py`, try the analogy `("tokyo", "japan", "paris")`. Does it return "france"? Try `("walking", "walked", "running")`. What does it return?
4. Compute the average cosine similarity between 500 random pairs of GloVe vectors. Is the space isotropic (are vectors spread evenly in all directions)?

Summary

- An **embedding** maps each token ID to a d-dimensional vector—a point in a high-dimensional space.
- The **dot product** $\mathrm{u}\cdot\mathrm{v} = \sum_i u_i v_i$ measures alignment between vectors. Divided by their lengths, it gives **cosine similarity**: a value between -1 and $+1$.
- The embedding matrix $\mathrm{W}_E \in \mathbb{R}^{|V|\times d}$ is learned entirely from training data. Its rows move to reflect which tokens appear in similar contexts.
- The **softmax** function turns a vector of raw scores into a probability distribution: $\mathrm{softmax}(\mathrm{z})_i = e^{z_i} / \sum_j e^{z_j}$.
- The learned geometry is rich: related words cluster, analogy arithmetic works, and most of the useful structure lives in a much lower-dimensional subspace than d.
- In the next chapter we solve the positional problem: the model currently has no way to know whether "cat sat on mat" or "mat sat on cat". We fix that.

4 Positional Encoding: Teaching the Model Where It Is

After this chapter you will understand why the Transformer has no built-in sense of order, how sinusoidal encodings inject position into the model, and how the modern RoPE method encodes relative *position directly into the attention calculation.*

We now have a sequence of embedding vectors—one per token. Each vector is a rich point in a high-dimensional space encoding the meaning of its token. But there is a problem we have quietly ignored: we have lost the order of the words.

Consider these two sentences:

The dog bit the man. vs *The man bit the dog.*

Same five tokens, completely different meanings. If the model receives the same five embedding vectors in any order and treats them as a *set*, it will produce the same output for both sentences. This would be catastrophic.

This chapter is about fixing that.

4.1 Why attention is order-blind

In Chapter 4 we will study attention in full detail. For now, just know this key fact: **attention treats its input as a set, not a sequence**. It com-

putes interactions between every pair of tokens. But it does not care which token came first and which came second—the mathematical operations are symmetric with respect to position.

This is intentional: it means the model can look at any position in relation to any other position without restriction. But it means that position information must be injected *separately*.

4.2 What we want from a positional encoding

We want to add something to each token's embedding vector that tells the model where that token is in the sequence. Call the position encoding of position t the vector p_t. We want:

1. **Unique:** each position t gets a different vector p_t.
2. **Deterministic:** we do not want to learn p_t from scratch—we want a fixed formula that works at any position.
3. **Translation-aware:** the relationship between position t and position $t + k$ should depend only on the gap k, not on the absolute values of t and $t + k$. This is important for the model to generalise: knowing that "the word three steps earlier was X" is more useful than knowing "the word at absolute position 17 was X".
4. **Bounded:** the values should not grow without limit as the sequence gets longer.

No single real number satisfies all four requirements. But there is a beautiful solution using waves.

4.3 Sine and cosine: the natural solution

4.3.1 A quick review of sine and cosine

If you have met sine and cosine before, skim to §3.3.2.

Imagine a point moving around a circle of radius 1 at constant speed. At time t, the point is at coordinates $(\cos t, \sin t)$.

- The horizontal position oscillates between -1 and $+1$.
- The vertical position also oscillates between -1 and $+1$.
- They oscillate at the same speed, but offset by a quarter-cycle.

- Both are bounded (never larger than 1 or smaller than -1).

The key property we will use: *sine and cosine at a fixed frequency ω are periodic*. They complete one full cycle every $2\pi/\omega$ steps.

4.3.2 Sinusoidal positional encoding

The original Transformer paper (2017) proposed this formula. For a sequence position t (starting at 0) and embedding dimension index i (starting at 0):

$$\mathrm{PE}(t,\ 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right)$$

$$\mathrm{PE}(t,\ 2i+1) = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

where d is the embedding dimension, and $i = 0, 1, 2, \ldots, d/2 - 1$.

Let's decode this carefully.

The fraction $10000^{2i/d}$ is what controls the frequency. For $i = 0$ (the first two dimensions), the frequency is $1/10000^0 = 1$—one full oscillation per 10,000 positions. Wait, that's wrong—let me be precise. The denominator is $10000^{2i/d}$, so:

- When $i = 0$: denominator $= 10000^0 = 1$, so the argument is just t. This oscillates quickly (one full cycle every $2\pi \approx 6.28$ positions).
- When $i = d/4$: denominator $= 10000^{0.5} = 100$, so the argument is $t/100$. Much slower.
- When $i = d/2-1$: denominator ≈ 10000, so the argument is $t/10000$. The slowest oscillation.

So the positional encoding uses many different frequencies simultaneously. The first pair of dimensions oscillates very rapidly; the last pair oscillates very slowly. Together they give a unique "signature" to every position—like a clock with many hands at different speeds.

What this really means

Think of a mechanical clock. The second hand completes a full revolution every 60 seconds (fast). The minute hand completes one every 60 minutes (slow). The hour hand once every 12 hours (very slow). From the positions of all three hands you can tell the exact time. The sinusoidal positional encoding works the same way: many hands at different speeds, together uniquely identifying each position.

4.3.3 Why the dot product property matters

Recall from the list of requirements: the relationship between position t and $t + k$ should depend only on k. Let's check this.

If we compute $\mathrm{PE}(t) \cdot \mathrm{PE}(t + k)$—the dot product of the encoding at position t with the encoding at position $t + k$—what do we get?

For each frequency $\omega_i = 1/10000^{2i/d}$, the i-th pair of dimensions contributes:

$$\sin(\omega_i t)\sin(\omega_i(t + k)) + \cos(\omega_i t)\cos(\omega_i(t + k))$$

Using the trigonometric identity $\cos(A - B) = \cos A \cos B + \sin A \sin B$, this simplifies to:

$$\cos(\omega_i t - \omega_i(t + k)) = \cos(-\omega_i k) = \cos(\omega_i k)$$

The t cancelled out completely. The dot product depends only on k.

$$\mathrm{PE}(t) \cdot \mathrm{PE}(t + k) = \sum_{i=0}^{d/2-1} \cos(\omega_i k)$$

This depends only on the gap k, not on the absolute position t.

This means the model can use dot products between positional encodings to compute how far apart two positions are, regardless of where in the sequence they appear.

4.4 Learned positional embeddings

An alternative: instead of using a fixed formula, just store a matrix $\mathrm{P} \in \mathbb{R}^{T_{\max} \times d}$ of position vectors and let the model learn them like any other parameters.

GPT-2 and BERT use this approach. The position 0 embedding is a learned vector, the position 1 embedding is a different learned vector, and so on up to some maximum sequence length $T_{\max}$.

Pros and cons:

+ Can adapt to the actual statistics of the training data—no need for a hand-crafted formula.
+ May work slightly better in practice on in-distribution lengths.
– Cannot generalise to sequences longer than $T_{\max}$. At position $T_{\max} + 1$, there is no learned vector.
– Adds $T_{\max} \times d$ parameters to the model—not huge but nonzero.

4.5 Rotary Positional Embedding (RoPE)

The most widely used positional encoding in current models is called **RoPE** (Rotary Position Embedding). Its key idea is different from everything above: instead of adding a positional vector to the token embedding before attention, we *rotate* the query and key vectors inside the attention computation. This way, position is encoded in how the model attends, not in the embeddings themselves.

RoPE is used in LLaMA, Mistral, Falcon, Gemma, and most major open-source models released after 2022.

4.5.1 The 2D rotation idea

Start in two dimensions. Given a 2D vector $\mathrm{x} = (x_1, x_2)$, its rotation by angle θ is:

$$\mathrm{R}_\theta \mathrm{x} \;=\; \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \;=\; \begin{pmatrix} x_1 \cos\theta - x_2 \sin\theta \\ x_1 \sin\theta + x_2 \cos\theta \end{pmatrix}$$

There is a crucial property of rotations: they preserve the dot product. If you rotate both u and v by the same angle θ, their dot product does not change:

$$(\mathrm{R}_\theta \mathrm{u}) \cdot (\mathrm{R}_\theta \mathrm{v}) \;=\; \mathrm{u} \cdot \mathrm{v}$$

This follows because rotations are orthogonal transformations (they preserve lengths and angles).

But here is the key: what if you rotate by *different* angles? If u is at position m and we rotate it by $m\theta$, and v is at position n and we rotate it by $n\theta$, then:

$$\begin{aligned}(\mathrm{R}_{m\theta}\mathrm{u}) \cdot (\mathrm{R}_{n\theta}\mathrm{v}) &= \mathrm{u}^\top \mathrm{R}_{m\theta}^\top \mathrm{R}_{n\theta}\mathrm{v} \\ &= \mathrm{u}^\top \mathrm{R}_{(n-m)\theta}\mathrm{v}\end{aligned}$$

The result depends on *only the difference* $n - m$. That is exactly the relative position.

> If we rotate query q at position m by angle $m\theta$, and key k at position n by angle $n\theta$, then:
>
> $$(\mathrm{R}_{m\theta}\mathrm{q}) \cdot (\mathrm{R}_{n\theta}\mathrm{k}) \;=\; \mathrm{q}^\top \mathrm{R}_{(n-m)\theta}\mathrm{k}$$
>
> The dot product—which drives attention—depends only on the relative offset $(n - m)$, not on absolute positions m and n separately.

4.5.2 Extending to d dimensions

For a d-dimensional embedding, we split the dimensions into $d/2$ pairs and apply a 2D rotation to each pair, using a different frequency θ_i for each pair:

$$\theta_i \;=\; \frac{1}{10000^{2i/d}}, \qquad i = 0, 1, \ldots, d/2 - 1$$

(The same geometric sequence as the sinusoidal encoding.) For position m, pair i gets rotated by angle $m\theta_i$.

The rotation matrix for all d dimensions is block-diagonal: $d/2$ tiny 2×2 rotation blocks, one per pair, on the diagonal and zeros everywhere else. In practice, we never form this large matrix—we apply each 2×2 block separately.

4.5.3 Why RoPE is the current standard

- **Relative by design:** attention naturally computes relative positions. There is no approximation.

- **No extra parameters:** unlike learned PE, RoPE adds nothing to the model.
- **Length extrapolation:** with some frequency scaling tricks (NTK-aware scaling, YaRN), RoPE-based models can handle sequences longer than they were trained on. Learned PE and sinusoidal PE cannot do this.
- **Efficient:** applying the rotation is just a few multiply-add operations per attention head.

4.6 Python: seeing positional encodings

ch3_pe_heatmap.py

```python
# Generate the sinusoidal PE matrix and visualise it as a heatmap.
# Shows: each column oscillates at a different frequency.
# Install: pip install numpy matplotlib

import numpy as np
import matplotlib.pyplot as plt

def sinusoidal_pe(T, d):
    """
    Build the sinusoidal positional encoding matrix.
    Returns an array of shape (T, d).
    T: number of positions (sequence length)
    d: embedding dimension (must be even)
    """
    PE = np.zeros((T, d))
    positions = np.arange(T)[:, None]          # shape (T,1)
    dim_pairs  = np.arange(0, d, 2)[None, :]   # shape (1, d/2)

    # Frequencies: 1 / 10000^(2i/d)
    freqs = 1.0 / (10000 ** (dim_pairs / d))   # shape (1, d/2)

    # Arguments to sin and cos: position × frequency
    angles = positions * freqs                  # shape (T, d/2)
```

```python

    PE[:, 0::2] = np.sin(angles)  # even indices
    PE[:, 1::2] = np.cos(angles)  # odd indices
    return PE

T = 100   # sequence length
d = 64    # embedding dimension

PE = sinusoidal_pe(T, d)
print(f"PE matrix shape: {PE.shape}")
print(f"Value range: [{PE.min():.3f}, {PE.max():.3f}]")

# ── Heatmap ──────────────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Full heatmap
im = axes[0].imshow(PE, aspect="auto", cmap="RdBu_r",
                    vmin=-1, vmax=1, origin="upper")
axes[0].set_xlabel("Embedding dimension")
axes[0].set_ylabel("Position in sequence")
axes[0].set_title("Sinusoidal PE  (T=100, d=64)")
plt.colorbar(im, ax=axes[0], shrink=0.8)

# Show a few individual dimensions
for col, color, label in [
        (0,  "#2C3E6B", "dim 0 (fastest)"),
        (8,  "#5c8ae0", "dim 8"),
        (32, "#9A6B00", "dim 32"),
        (62, "#e05c5c", "dim 62 (slowest)")]:
    axes[1].plot(PE[:, col], color=color, linewidth=1.5,
     ↪  label=label)

axes[1].set_xlabel("Position in sequence")
axes[1].set_ylabel("Encoding value")
axes[1].set_title("Individual dimensions of PE")
axes[1].legend(fontsize=8)
axes[1].spines[["top","right"]].set_visible(False)

plt.tight_layout()
plt.savefig("figures/ch3_pe_heatmap.pdf", bbox_inches="tight")
plt.show()
print("Figure saved to figures/ch3_pe_heatmap.pdf")
```

ch3_pe_dotproduct.py

```python
# Verify that PE(t) · PE(t+k) depends only on k, not on t.
# This is the translation-invariance property.

import numpy as np
import matplotlib.pyplot as plt

# Re-use the function from above (or paste it here)
# PE = sinusoidal_pe(T, d)   — assumed already defined

T = 200
d = 128
PE = sinusoidal_pe(T, d)

# Compute PE(t) · PE(t+k) for several values of t
k_values = np.arange(0, 80)
test_positions = [0, 20, 50, 100]

fig, ax = plt.subplots(figsize=(8, 4.5))

for t0 in test_positions:
    dots = [np.dot(PE[t0], PE[t0 + k]) for k in k_values
            if t0 + k < T]
    ks   = k_values[:len(dots)]
    ax.plot(ks, dots, linewidth=1.8,
            label=f"t = {t0}", alpha=0.85)

ax.set_xlabel("Offset k")
ax.set_ylabel("PE(t) · PE(t+k)")
ax.set_title("Dot product depends only on offset k,\n"
             "not on absolute position t  (curves overlap)")
ax.legend(fontsize=9)
ax.spines[["top","right"]].set_visible(False)
plt.tight_layout()
plt.savefig("figures/ch3_dotproduct.pdf", bbox_inches="tight")
plt.show()
print("Figure saved to figures/ch3_dotproduct.pdf")
```

When you run this, you should see that all four curves nearly overlap. The dot product at offset k is nearly the same regardless of whether $t = 0$ or $t = 100$. (Not perfectly identical due to finite d, but very close.) This is the translation-invariance property proven in §3.3.3—visualised.

ch3_rope.py

```python
# Implement RoPE from scratch and verify the relative-position property.
# No external libraries beyond numpy.

import numpy as np

def rotate_half(x):
    """
    Given x of shape (..., d), returns a rotated version where
    each adjacent pair (x[2i], x[2i+1]) becomes (-x[2i+1], x[2i]).
    This implements the 2D rotation without building a block matrix.
    """
    x1 = x[..., 0::2]   # even indices
    x2 = x[..., 1::2]   # odd indices
    return np.concatenate([-x2, x1], axis=-1)

def apply_rope(x, position, d):
    """
    Apply RoPE to vector x at given position.
    x: shape (d,)
    Returns rotated vector of the same shape.
    """
    # Frequencies: theta_i = 1 / 10000^(2i/d)
    i      = np.arange(d // 2)
    thetas = 1.0 / (10000 ** (2 * i / d))    # shape (d/2,)

    # Angles for this position
    angles = position * thetas                # shape (d/2,)

    # Interleave: angles for pairs (0,1), (2,3), (4,5), ...
    cos_a = np.repeat(np.cos(angles), 2)      # shape (d,)
    sin_a = np.repeat(np.sin(angles), 2)      # shape (d,)

    return x * cos_a + rotate_half(x) * sin_a

d = 64   # embedding dimension (must be even)

# —— Test the relative-position property ——————————————
# Create random query and key vectors (simulating one attention head)
```

```python
np.random.seed(42)
q = np.random.randn(d)
k = np.random.randn(d)

print("=" * 55)
print("Testing RoPE relative-position property")
print("=" * 55)
print()
print("Claim: dot(RoPE(q, m), RoPE(k, n)) depends only on (n-m)")
print()

# Test 1: different absolute positions, same relative offset
offset = 7
pairs = [(0, 7), (10, 17), (50, 57), (100, 107)]

print(f"Relative offset = {offset}  (fixed)")
print(f"{'Positions (m,n)':<22} {'dot product':>14}")
print("-" * 38)
for m, n in pairs:
    q_rot = apply_rope(q, m, d)
    k_rot = apply_rope(k, n, d)
    dot   = np.dot(q_rot, k_rot)
    print(f"  ({m:3d}, {n:3d})             {dot:>14.6f}")

print()
print("↑ All values should be nearly identical (same offset = same dot product)")
print()

# Test 2: same absolute positions, different relative offsets
print("Position m=0 fixed, varying n:")
print(f"{'n':<8} {'offset (n-m)':<16} {'dot product':>12}")
print("-" * 38)
q_rot = apply_rope(q, 0, d)
for n in [0, 1, 3, 7, 15, 31]:
    k_rot = apply_rope(k, n, d)
    dot   = np.dot(q_rot, k_rot)
    print(f"  {n:<6} {n:<16} {dot:>12.6f}")

print()
```

```
print("↑ Different offsets give different dot products — position is encoded.")
```

Run this and check the first table. All four rows should give almost exactly the same dot product. This confirms what the math predicted: the dot product depends only on the gap, not on where in the sequence you are. The second table shows that different gaps give different dot products—position is indeed encoded.

ch3_pe_compare.py

```
# Compare GPT-2's learned PE matrix with sinusoidal PE via PCA.
# Shows: learned PE recovers approximately sinusoidal structure.
# Install: pip install transformers torch numpy matplotlib scikit-learn

import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from transformers import GPT2Model

# ── Load GPT-2 learned positional embeddings ──────────────
model = GPT2Model.from_pretrained("gpt2")
model.eval()

# wpe: word position embeddings, shape [1024, 768]
learned_PE = model.wpe.weight.detach().numpy()
T_max, d   = learned_PE.shape
print(f"GPT-2 learned PE shape: {learned_PE.shape}")

# ── Build sinusoidal PE of same dimensions ────────────────
def sinusoidal_pe(T, d):
    PE = np.zeros((T, d))
    pos  = np.arange(T)[:, None]
    dims = np.arange(0, d, 2)[None, :]
    freq = 1.0 / (10000 ** (dims / d))
    ang  = pos * freq
    PE[:, 0::2] = np.sin(ang)
    PE[:, 1::2] = np.cos(ang)
    return PE

```

```python
sin_PE = sinusoidal_pe(T_max, d)

# ── PCA: project to 2D ──────────────────────────────────
pca    = PCA(n_components=2)
pca.fit(np.vstack([learned_PE, sin_PE]))   # fit on combined data

L_2d = pca.transform(learned_PE)           # learned PE in 2D
S_2d = pca.transform(sin_PE)               # sinusoidal PE in 2D

# ── Plot ────────────────────────────────────────────────
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.5))

for ax, pts, title, color in [
        (ax1, L_2d, "GPT-2 learned PE\n(positions 0–1023 as a path)",
         "#2C3E6B"),
        (ax2, S_2d, "Sinusoidal PE\n(same shape!)", "#9A6B00")]:
    sc = ax.scatter(pts[:, 0], pts[:, 1],
                    c=np.arange(T_max), cmap="viridis",
                    s=6, alpha=0.7)
    ax.plot(pts[:, 0], pts[:, 1], linewidth=0.4,
            color=color, alpha=0.4)
    ax.set_title(title, fontsize=10)
    ax.set_xlabel("PC1")
    ax.set_ylabel("PC2")
    ax.spines[["top","right"]].set_visible(False)

# Shared colorbar
cbar = plt.colorbar(sc, ax=[ax1, ax2], shrink=0.7, pad=0.02)
cbar.set_label("Position index", fontsize=9)

plt.suptitle("Learned vs sinusoidal PE in 2D — both trace similar spirals",
             fontsize=11)
plt.tight_layout()
plt.savefig("figures/ch3_pe_compare.pdf", bbox_inches="tight")
plt.show()
print("Figure saved to figures/ch3_pe_compare.pdf")
```

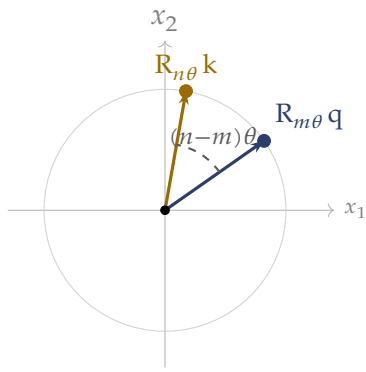
Both panels should show a spiral or winding curve, with the colour

gradually shifting from dark (early positions) to light (late positions). The fact that the learned PE (which was never told to look like a sinusoid) traces nearly the same shape as the hand-designed sinusoidal PE is striking. It suggests that this spiral structure is in some sense the *right* way to represent sequential position.

1. In `ch3_pe_heatmap.py`, change `T = 100` to `T = 500`. Do the slow-frequency columns (right side of the heatmap) complete even one full oscillation? What does that tell you about how long sequences need to be before those dimensions become useful?
2. In `ch3_rope.py`, change the `10000` base to `500`. Now re-run the first test (same offset, different absolute positions). Are the dot products still the same? Why? Then re-run the second test (different offsets). Do the values now change faster with offset?
3. In `ch3_pe_dotproduct.py`, increase `d` from 128 to 512. Do the curves overlap more or less? Why does higher dimension improve the approximation?
4. In `ch3_pe_compare.py`, try looking only at the first 100 positions (slice `learned_PE[:100]` and `sin_PE[:100]`). Does the spiral structure become clearer?

Summary

- Attention is order-blind. Without extra help, the model cannot distinguish "dog bites man" from "man bites dog".
- We need positional encodings satisfying four properties: unique, deterministic, translation-aware, and bounded.
- **Sinusoidal PE** uses sine and cosine at many different frequencies. The dot product $\mathrm{PE}(t) \cdot \mathrm{PE}(t+k)$ depends only on the offset k, as proven by the $\cos(A - B)$ identity.
- **Learned PE** (used in GPT-2, BERT) stores a trainable matrix of position vectors. Flexible but cannot extrapolate.



$(\mathrm{R}_{m\theta}\mathrm{q}) \cdot (\mathrm{R}_{n\theta}\mathrm{k})$ depends only on $(n-m)\theta$

Figure 4.1: RoPE in 2D. The query vector q at position m is rotated by angle $m\theta$; the key k at position n is rotated by angle $n\theta$. Their dot product depends only on the difference $(n-m)\theta$—the relative offset. Real RoPE applies this to $d/2$ independent pairs of dimensions simultaneously.

- **RoPE** rotates query and key vectors inside attention by angles proportional to their positions. The dot product then automatically encodes relative position. This is the current standard.
- Now that each token has both a meaning vector (embedding) and a position vector, we are ready to build the mechanism that lets tokens communicate: **attention**.